Product-Level Demand Forecasting for Ingredient Requirement Estimation to Reduce Food Waste

Eduardo Heras Barrales

3138899

Submitted in partial fulfilment for the degree of

Master of Big Data Management and Analytics

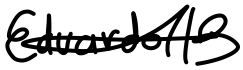
Griffith College Dublin

September, 2026

Under the supervision of Ahmed Makki

Disclaimer

I hereby certify that this material, which I now submit for assessment on the program of study leading to the Degree of Master of Big Data Management and Analytics at Griffith College Dublin, is entirely my own work and has not been submitted for assessment for an academic purpose at this or any other academic institution other than in partial fulfillment of the requirements of that stated above.

Signed:  **Date: 08 September 2026**

Acknowledgements

I would like to take this opportunity to thank the people who supported me throughout the development of this thesis.

- Ahmed Makki: For his guidance, advice and support as my supervisor throughout this thesis. His feedback and direction were invaluable in shaping this work.
- Griffith College Dublin: For providing the education, resources and environment that made this research possible.
- Family and friends: For their patience and constant support throughout this process. Their encouragement, even in the smallest ways, made this journey much easier to navigate.

Contents

1 Introduction **10**

2 Background **12**

2.1 Extra Features 12

2.2 Models 14

3 Methodology **17**

3.1 Recursive forecasting 17

3.2 Dashboard 18

3.3 Methodology 18

3.4 Scalability Considerations 19

3.5 Summary 20

4 System Analysis and Design **22**

4.1 EDA 22

4.1.1 Dataset Overview 22

4.1.2 Data Cleaning Process 23

4.1.3 Seasonality 24

4.1.4 Ingredients 26

4.1.5 Weather Feature 26

4.1.6 Lag and Cyclical Features 28

4.1.7 Correlation 29

4.1.8 Demand Intermittency and Sparsity 30

4.2 Modeling and comparison 31
4.2.1 LSTM Model 31
4.2.2 XGBoost Model 32
4.2.3 Prophet Model 33
4.2.4 Random Forest Model 33
4.2.5 Baseline Models 33
4.2.6 Summary 34
4.3 Forecast and Ingredients 37
4.3.1 Confidence Levels 37
4.3.2 Ingredients 38

5 Implementation **40**
5.1 Pipeline 40
5.1.1 Orchestrator and Configuration 40
5.1.2 Data Ingestion (Step 2) 41
5.1.3 Feature Engineering and Forecasting (Steps 3-4) 41
5.1.4 Forecast PostgreSQL Table and Final Excel Output (Steps 5-6) 42
5.1.5 Summary 43
5.2 Dashboard 43
5.2.1 Configuration 44
5.2.2 Page 1 44
5.2.3 Page 2 45
5.2.4 Summary 45

6 Testing and Evaluation **46**
6.1 Weather 46
6.2 Models 47
6.3 Practical Implications of Ingredient-Level Requirement Estimation 49
6.4 Summary 49

7 Conclusion **51**

7.1 Future Work 52

A Appendix **57**

List of Figures

4.1 Boxplot of orders per variant 24
4.2 Sales per weekday 25
4.3 Average sales per weekday per month 25
4.4 Ingredients per pizza 26
4.5 Daily Average Temperature vs Precipitation 27
4.6 Monthly Orders vs Average Weather Score 28
4.7 Correlation Matrix — Sales and Candidate Features 29
4.8 Distribution of mean daily demand across products 31
4.9 Models Comparison 34
4.10 Error per model 35
4.11 XGBoost model top features 36

A.1 Models Comparison 58
A.2 Error per model 58
A.3 Data pipeline flow, from raw order ingestion to the interactive dashboard. 59

List of Tables

4.1 Cross-distribution of category and size (variant) 23
4.2 Mean error and Standard deviation for each Horizon day 38

Abstract

Food waste in the restaurant sector represents a significant economic and environmental problem, whose root cause is frequently attributed to poor purchasing and inventory planning derived from inaccurate demand forecasting. Most existing systems address this problem at the finished product level, leaving a relevant gap between the level at which forecasting is performed and the level at which purchasing and inventory decisions actually occur: the individual ingredient. This thesis proposes and implements a complete product-level demand forecasting system whose output is translated into ingredient-level requirements, using a pizzeria as a case study, with the objective of reducing food waste through more precise input planning. A complete data pipeline was created using a series of historical orders, and four different forecasting methods were tested on an empirical basis and benchmarked against simple historical-mean and seasonal-naive baselines, confirming the predictive value of the proposed variables such as calendar variables, lag features, and a composite weather index. Since XGBoost provided performance comparable to others, it was chosen as the production model. The system produces a three-day recursive forecast together with empirical confidence intervals, and the ingredient requirements are derived from this forecast by means of a relational catalog. The system is complemented by an interactive dashboard that converts the output of the pipeline into useful information for non-technical staff, showing that translating product-level forecasts into ingredient requirements can be effectively used as a direct measure to combat food waste.

Chapter 1

Introduction

Demand forecasting in the food and restaurant sector presents particularities that distinguish it from demand forecasting in other retail sectors, mainly due to the perishability of inputs, the high variability in the combination of products sold, and replenishment cycles considerably shorter than those of non-perishable goods [10]. These particularities have motivated research on forecasting methods applied to this sector, ranging from classical statistical approaches to more recent machine learning models [10]. This work aims to evaluate whether a set of proposed variables, related to historical sales behavior, calendar patterns, and weather conditions, effectively provides predictive value in this context, rather than assuming their relevance solely based on their use in other studies. Food waste is one of the most significant economic and environmental problems of the global food supply chain, with estimates placing a considerable proportion of total food production as wasted before consumption [9]. The restaurant and food service sector, in particular, contributes substantially to this figure, as it operates under conditions of high uncertainty regarding daily demand, combined with the perishable nature of its inputs [9]. Among the root causes of food waste in restaurants, the literature recurrently identifies poor purchasing and inventory planning, typically derived from inaccurate or nonexistent demand forecasting, as one of the most determining factors, above other causes such as

inadequate portion handling or preparation losses [9]. This positions demand forecasting not as just another operational optimization tool, but as a direct intervention in one of the main causes of the problem. However, the literature also points to a relevant limitation in the way this intervention has historically been approached: the vast majority of forecasting systems implemented in the sector operate at the finished product level, while the purchasing and inventory decisions that effectively determine waste occur at the individual input or ingredient level. This disconnect between the level at which forecasting is performed and the level at which purchasing and storage occur constitutes, precisely, the central motivation of this thesis: an accurate product-level forecast can still generate waste if it is not correctly translated into specific requirements for each ingredient, as the same forecasting error can be amplified or offset in non-obvious ways when aggregated across multiple products that share inputs. In this sense, the present work does not propose merely a sales forecasting system, but an intervention specifically directed at the level of granularity at which the literature identifies the most relevant gap between past research and the root cause of food waste in restaurants.

Chapter 2

Background

Before evaluating the relevance of any candidate variable for the forecasting model, it is necessary to ensure the quality of the dataset from which such variables are built. For this reason, the process followed in this project will comprise two sequential stages: first, a data cleaning stage that ensures the integrity of the training set; and second, the construction of the features that feed the evaluated models. The literature on applied machine learning for the cleaning process of a dataset, agrees that the quality of the input data, the absence of duplicates, the appropriate treatment of missing values, and the identification of outliers is a prerequisite for the validity of any subsequent model, regardless of its sophistication [15].

2.1 Extra Features

Exclusion of low-volume categories. The exclusion of product categories or variants with marginal statistical representation, as occurred in this work with the XL and XXL sizes, is used in the retail demand forecasting literature, same as the treatment of long-tail items, where insufficient historical data volume can negatively affect the generalization ability of a model trained jointly with high-turnover products [6]

Once the quality of the dataset has been ensured, the next step consists of defining which variables have the potential to provide predictive value to the model. The following presents the theoretical justification for each of the features proposed for this project, evaluating their support in the literature before determining, empirically in later chapters, whether such support is confirmed in the case study analyzed.

- **Lag features** The use of past values of a time series as predictor variables for its future value is one of the most established principles in time series forecasting, because most demand phenomena exhibit some degree of autocorrelation with their own recent values [13]. Justifying the inclusion of extra features like `lag_1` and `lag_7` as core features of the proposed model.

- **Cyclical encoding of temporal variables.** Representing categorical temporal variables (day of the week, month) through trigonometric functions (sine and cosine) instead of simple categorical encoding is a a common approach for preserving the periodic nature of these patterns for models that would otherwise treat numerically distant values (for example, December and January) as unrelated to one another [12].

- **Calendar effects** A variable called `weekend_holiday` was included as a core feature, bacause the literature on demand forecasting in retail and restaurants consistently reports the existence of calendar effects —systematic variations associated with weekends and holidays, as a relevant factor to be explicitly incorporated into forecasting models [16].

- **Weather variables** [2] shows how composite indices that combine multiple meteorological variables (for example, temperature and precipitation) into a single measure, rather than incorporating each variable independently, give a more accurate weather score. Arguing that this combination more directly captures the weather conditions on consumption behavior. The weighting scheme for combining temperature and precipitation into a weather index was set at 60% for temperature and 40% for precipita-

tion. Temperature was given more weight because it is thought to have a stronger effect on food consumption. These proportions were chosen based on practical judgment, not on previous empirical studies.

The literature is not unanimous regarding which external contextual variables (weather, events, payday, etc.) are effectively relevant for demand forecasting: while some studies report significant relationships, others find weak or context-dependent effects (depending on product type, sales channel, geographic location) [2]. This lack of consensus methodologically motivates the approach adopted in this work: rather than assuming beforehand that a variable such as weather will be relevant, its importance is evaluated empirically through correlation analysis and feature importance analysis of the trained model, so that the data itself determines its actual contribution to the forecast.

2.2 Models

With the variables already defined and justified, the next step consists of selecting the modeling approaches to be evaluated for the forecasting task. The following presents the theoretical justification for each of the models considered in this project, contrasting their characteristics and supporting the decision to compare them against one another before determining, empirically, which offers the best balance between performance, computational cost, and interpretability for this case study.

- **LSTM networks** Long Short-Term Memory (LSTM) recurrent neural networks have been widely used for time series forecasting due to their ability to capture medium- and long-term temporal dependencies through internal memory mechanisms, outperforming classical statistical models such as ARIMA in several contexts [17].

  Embeddings for categorical variables. When a model must process multiple related time series (for example, different products) jointly, the liter-

ature suggests use of embedding layers to represent categorical identifiers through learned dense vectors, instead of one-hot encoding, allowing the model to capture similarities between entities with similar demand patterns [8].

- **Gradient boosting models (XGBoost)** Gradient boosting models over decision trees, such as XGBoost, have demonstrated competitive performance against deep learning approaches in forecasting tasks with tabular data, with the additional advantage of requiring a smaller volume of training data and lower computational cost [10].

- **Random Forest** Random Forest ensemble models are frequently used as a baseline for comparison against boosting methods, as they share the same family of tree-based models, allowing isolation of whether observed performance improvements stem specifically from the boosting algorithm or from the use of decision trees in general [1].

- **Prophet** Prophet is a forecasting model that decomposes a time series into trend and seasonality components, fitted independently for each series, designed to be robust to missing data and trend changes, although generally less flexible than machine learning approaches in capturing complex relationships among multiple predictor variables [18].

The choice among different modeling approaches should not be based on error metrics, but should also consider the computational cost and interpretability of each model, particularly in systems intended for actual operational use by non-technical users, where the ability to explain a prediction can hold practical value equivalent to or greater than a marginal improvement in accuracy [10]. The vast majority of the literature on demand forecasting in restaurants and retail operates at the product level, with considerably less attention devoted to forecasting at the level of an individual component or ingredient. This gap is relevant for restaurants with product catalogs that share ingredients, where a forecast aggregated only at the product level can obscure the true demand

exposure of a specific input. Hierarchical forecasting approaches developed in adjacent domains offer conceptual precedents for how to address granularity in forecasting, although they are rarely applied directly to the ingredient-level context in the restaurant sector [16]. This thesis is positioned precisely at that underexplored intersection.

Chapter 3

Methodology

3.1 Recursive forecasting

For short forecasting horizons, the strategy consists of generating predictions recursively, using the prediction of one period as input for the next, rather than training an independent model for each step of the horizon [13], strategy used for the three-day forecast. As an alternative to assuming a theoretical distribution (for example, normal) for a model's errors, the literature tells how the construction of confidence intervals empirically, based on the distribution of errors observed directly on a validation or test set [3]. Following this idea the estimate the confidence margin of the three-day forecast was calculated. To evaluate how well the models are trained, the Mean Absolute Error (MAE) and the Root Mean Squared Error (RMSE) are used, which are among the most widely used metrics for evaluating and comparing demand forecasting models, because they express the error in the same units as the forecasted variable, which facilitates its practical interpretation [11]. In addition the Weighted Absolute Percentage Error (WAPE) is incorporated as a complementary metric. Percentage-based error metrics are generally useful for expressing forecast accuracy in relative terms, independent of the scale of the target variable, which facilitates comparison across products with very different demand levels.

3.2 Dashboard

Beyond the performance and interpretability of the model, the literature on decision support systems points out that the practical value of a forecasting model largely depends on its results being accessible and actionable for the end user, specially when the latter does not have prior technical knowledge. In this sense, frameworks oriented toward the rapid development of interactive data applications, such as Streamlit, have been increasingly adopted for this purpose, as they allow the translation of machine learning model results into visual interfaces without requiring traditional web development knowledge, supporting the decision to implement this project's presentation layer as an interactive dashboard, rather than limiting the delivery of results to a static report. [20]

3.3 Methodology

The final objective is to create a data pipeline structured in sequential stages, ranging from the ingestion of raw data to the generation of an actionable product-level forecast translated into ingredient requirements. The general architecture includes the following components:

1. Distributed processing inspired by Map & Reduce, aimed at demonstrating the conceptual scalability of the system in the face of larger data volumes.

2. The use of a PostgreSQL relational database, selected for its robustness, support for referential integrity, and its suitability for modeling the relationships among orders, products, and ingredients.

3. A feature engineering module.

4. A data forecasting model.

5. A results generation layer oriented toward consumption by restaurant staff.

This architecture seeks to be generalizable to other food service contexts beyond the proposed case study, in line with the objective of proposing a replicable pipeline.

3.4 Scalability Considerations

The pipeline was implemented and validated on a scale suitable for a single-restaurant case study (48,048 orders, one location, one year), but the architecture was intentionally designed in such a way that each stage has a distributed equivalent without needing a redesign of the underlying logic. This distinction is important as setting up a real cluster for a dataset of this size would introduce unnecessary operational overhead without any corresponding benefit, the design choice, therefore, was to keep the current implementation single-node while ensuring that the natural growth path of the system — from a single restaurant to a chain of multiple locations — does not require re-architecting the pipeline from scratch.

The ingestion stage (Step 2), currently simulated locally through a subprocess-based Map & Reduce job over a single daily CSV file, is structurally equivalent to a Hadoop MapReduce or Apache Spark batch job reading partitioned files from a distributed file system (HDFS) or object storage, with each partition corresponding naturally to a restaurant location or a calendar day [19]. This partitioning strategy is what would allow the same ingestion logic to scale from one restaurant to hundreds without algorithmic changes, only a change of execution engine.

The feature engineering stage (Steps 3–4) groups records by an item id and computes lag and cyclical features independently per product. This operation is embarrassingly parallel by construction, since no product's features depend on another product's data, which makes it a direct candidate for a distributed Data Frame API such as Spark's, using grouping functions and window functions in place of the current pandas-based implementation. The same reasoning extends to a multi-location scenario, where features could additionally be partitioned

by restaurant, allowing the system to scale horizontally with the number of locations rather than only with the number of products.

Regarding the modeling stage, XGBoost itself natively supports distributed training across a cluster, meaning that the production model selected in this thesis would not need to be replaced if the training data grew by orders of magnitude, only its execution backend would change [4]. This reinforces one of the practical arguments for selecting XGBoost over other models, beyond its lower training cost and interpretability at the current scale, it is also the model best positioned to scale to a distributed setting without a change of algorithm.

Finally, the choice of PostgreSQL for persistence (Step 5) is appropriate at the current scale due to its support for referential integrity across orders, products, and ingredients. At a scale involving multiple locations and multi-year historical data, this relational layer could be complemented with a separate analytical storage layer (e.g., a columnar format such as Parquet on distributed storage, or a cloud data warehouse), following a common separation between an operational serving layer optimized for low-latency dashboard queries and a historical batch layer optimized for large-scale analytical processing [14]. The current single-database design is therefore a simplification, appropriate to the prototype's scale.

3.5 Summary

The chapter sets out the theoretical basis for each of the project's methodological choices: the importance of food waste as a key issue, the feature engineering techniques that were proposed (such as lags, cyclical encoding, calendar effects, and weather), the modeling approaches to be compared, and the multi-horizon forecasting strategy together with uncertainty quantification. It demonstrated that, even though the various components have been the subject of separate and established literature, they are rarely combined into one system that also translates its output into ingredient-level requirements. The project's general outline was also given herein, comprising the system architecture, the feature

engineering strategy, the comparative modeling approach, and the method for converting the forecasts into ingredient requirements. The subsequent chapters describe the implementation of the plan, findings from the exploratory data analysis, the results achieved by each model, and the design modifications carried out during the actual implementation of the system.

Chapter 4

System Analysis and Design

4.1 EDA

4.1.1 Dataset Overview

For the purpose of designing and implementing a demand forecasting system that contributes to reducing food waste through ingredient-level requirement estimation, a publicly available dataset found on Kaggle will be used.

This dataset contains approximately 48,000 historical sales orders (48,620 orders, to be precise), corresponding to 91 product-and-size combinations, and covers a full year, from January 1 to December 31, 2015 (365 days). Each order is described across 12 columns, including order and product identifiers, quantity, order date and time, unit and total price, size, category, ingredients and product name; since each ordered item can be associated with a recipe, this also yields an ingredient catalog. The dataset can be found at: `https://www.kaggle.com/datasets/shilongzhuang/pizza-sales`

Based on this data, it was decided to split the dataset into two tables: one being an order log (`orders`), which consists of four columns—the order id, the order date, the order quantity, and the `item_id` of the ordered product—where this `item_id` is mapped to the `product_catalog` table, in which each ID repre-

Category	Variant				
	L	M	S	XL	XXL
Chicken	6	6	6	0	0
Classic	7	7	8	1	1
Supreme	8	8	9	0	0
Veggie	9	8	7	0	0

Table 4.1: Cross-distribution of category and size (variant)

sents the aforementioned product and size.

The other being a product catalog (`product_catalog`) of 91 unique product-and-size combinations, corresponding to 32 distinct pizza names, distributed across 4 categories: Classic, Veggie, Supreme, and Chicken. Table 4.1 shows the cross-distribution of category and size (variant):

XL and XXL sizes are available only for the Classic category, which already anticipates their low representation in total sales volume. Similarly, the most and least used ingredients were calculated, where among the most used are Garlic, Tomatoes, and Red Onions, and among the least used are Brie Carre Cheese, Prosciutto, and Caramelized Onions. This offers a preview of what can be expected in the ingredient requirement estimation.

4.1.2 Data Cleaning Process

The data quality check on the orders table identified no missing values in any of its columns, nor any duplicate records. In the `product_catalog` catalog table, 65 unique ingredient names were identified. A manual review of this list allowed the detection of a misspelling in one name, which was corrected before continuing with the analysis.

It was identified that the XL and XXL size products were sold only 572 times throughout the entire year, approximately 1.2% of the total 48,620 records. Figure 4.1 visually confirms this low representation: while the L, M, and S sizes show a daily sales distribution with medians between 40 and 50 units, the XL

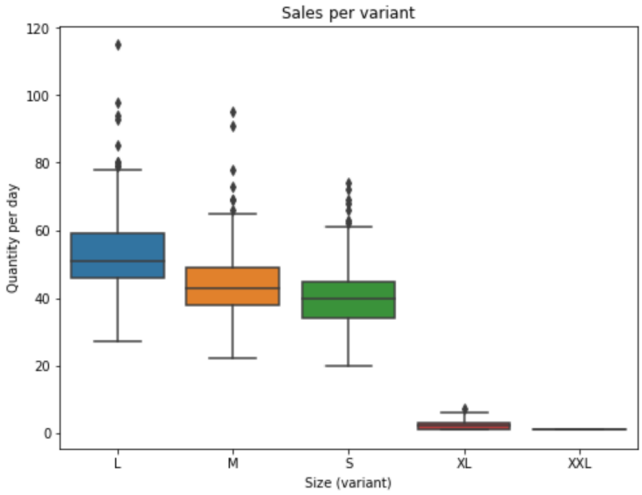


Figure 4.1: Boxplot of orders per variant

and XXL sizes are concentrated very close to zero, with minimal dispersion. These records were removed from the dataset used for modeling, reducing the total number of orders from 48,620 to 48,048. As a mitigation measure against the possibility that this type of order occurs in practice.

4.1.3 Seasonality

The seasonality analysis revealed a clear weekly pattern in order volume. As shown in figure 4.2, Fridays are the days with the most orders, above 8,000, followed by Thursday and Saturday, while Sunday has fewer than 6,000 orders, even below Monday. This pattern suggests that the highest demand is concentrated toward the end of the work week and the beginning of the weekend, rather than across the entire weekend.

To complement this, a heatmap crossing day of the week with month of the year was created, showing that Fridays consistently maintain the highest average order values throughout the year, with an additional intensification in the months of May, July, and November.

These patterns confirm the need to capture the weekly seasonality identified

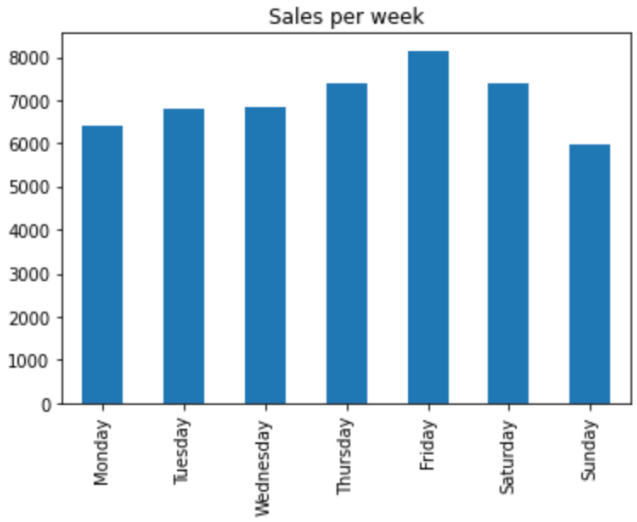


Figure 4.2: Sales per weekday

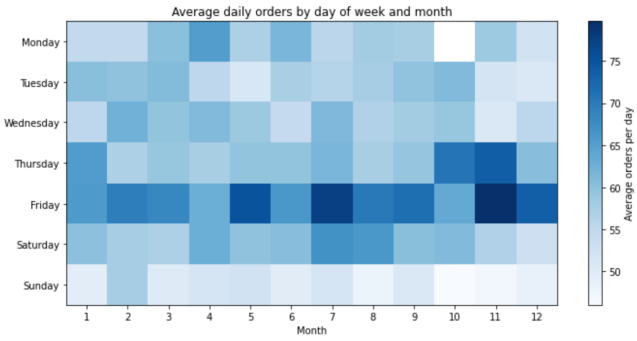


Figure 4.3: Average sales per weekday per month

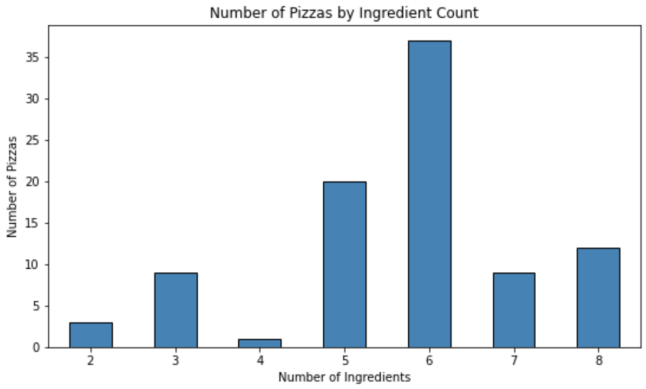


Figure 4.4: Ingredients per pizza

in the Methodology through cyclical encoding of the day of the week and the month, rather than treating them as simple categorical variables, as well as the relevance of the weekend/holiday flag (`weekend_holiday`) incorporated as an additional feature.

4.1.4 Ingredients

Thanks to Figure 4.4, it can be observed that most of the products in the catalog contain between 5 and 6 ingredients, with an average use of 5.69 ingredients per product. Therefore, it can be stated that there is a high level of ingredient reuse, which supports the idea that forecasting only at the product level can underestimate the true complexity of input planning. Forecasting errors from different products that share the same ingredient can accumulate or offset one another in ways that are not obvious if they are not explicitly considered and aggregated at the ingredient level.

4.1.5 Weather Feature

Another important feature is `weather_score`; as already justified, temperature and precipitation per day are needed in order to calculate it. Since the dataset

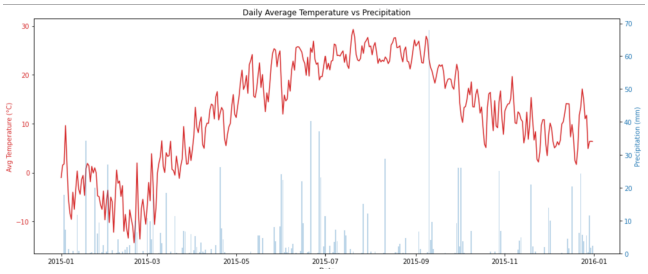


Figure 4.5: Daily Average Temperature vs Precipitation

corresponds to a pizzeria in New York, real historical temperature and precipitation data corresponding to the year 2015 were obtained through the Open-Meteo API. The code that performs this transformation was developed specifically for this project, since no standardized public implementation was identified for this type of composite index applied to the restaurant sector; the normalization and combination logic, however, follows the general principle validated in the literature consulted, even though the specific weights used correspond to a decision made by the author. Figure 4.5 shows the expected seasonal behavior: temperatures ranging from approximately -15°C in winter to 29°C in summer, with precipitation events irregularly distributed throughout the year.

Building the `weather_score` index on a scale from 0 (unfavorable condition) to 5 (favorable condition), weighting 60% temperature and 40% precipitation. This graph (Figure 4.6) compares the score against sales by month.

It is observed that, although the weather score varies markedly throughout the year (from values close to 1.3-1.5 in the winter months to 4.4-4.6 in July and August), the total monthly order volume remains relatively stable, without a growth or decline pattern clearly associated with the improvement or worsening of the weather. This observation is a relevant finding for the thesis's central research question: at the aggregate monthly level, no strong relationship is observed between favorable weather conditions and a higher volume of pizza sales. This finding does not invalidate the inclusion of the weather score as a feature in the modeling, since a weak relationship at the aggregate monthly level

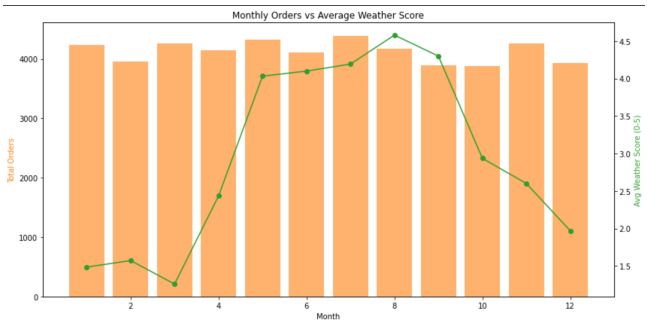


Figure 4.6: Monthly Orders vs Average Weather Score

does not rule out the existence of a more subtle relationship at the daily level or at the individual product level, which is quantitatively evaluated through the feature importance of the XGBoost model.

4.1.6 Lag and Cyclical Features

The lag features were built at the individual product level, grouping the data by `item_id` and sorting it chronologically, and then applying a shift over the quantity sold column: `lag_1` corresponds to the quantity sold on the immediately preceding day for that same product, while `lag_7` corresponds to the quantity sold on the same date the previous week. Since this calculation generates missing values during the first seven days of each product , because no previous value is available, these records were removed from the dataset before continuing with the analysis. For its part, cyclical encoding was applied to the `day_of_week` (with period 7) and `month` (with period 12) variables, transforming each one into a pair of sine and cosine components through the functions $\sin\left(\frac{2\pi x}{\text{period}}\right)$ and $\cos\left(\frac{2\pi x}{\text{period}}\right)$. This transformation makes it possible to represent, for example, that Sunday (day 6) and Monday (day 0) are close to each other in the feature space, something that a simple numerical encoding fails to capture correctly, as it would treat them as the most distant possible values within the range.

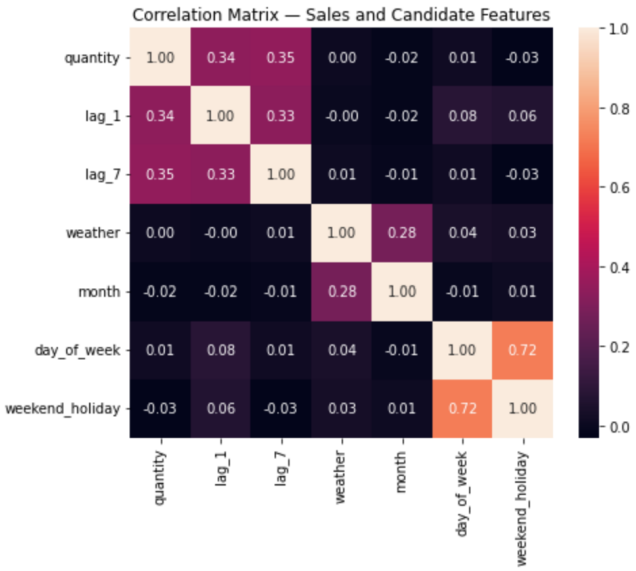


Figure 4.7: Correlation Matrix — Sales and Candidate Features

4.1.7 Correlation

Once the dataset was ready to be used, a Pearson correlation matrix (Figure 4.7) was created between the target variable (`quantity`) and the candidate features for modeling: `lag_1`, `lag_7`, `weather_score`, `month`, and `day_of_week`.

The lag features show a moderate, positive correlation with sales: 0.34 for `lag_1` and 0.35 for `lag_7`, indicating that both variables provide a useful predictive signal. However, the correlation between `lag_1` and `lag_7` (0.33) is moderate enough not to represent a relevant multi-collinearity problem. The weather variable shows a practically null correlation with `quantity` (0.00) at the daily level. This result is consistent with the finding already reported at the aggregate monthly level, where order volume remained stable regardless of variations in the weather score. Having two independent observations, one visual at the monthly level and another numerical at the daily level—pointing in the same direction reinforces the conclusion that, for this dataset, weather does not exhibit a direct linear relationship with sales volume. For their part, `weekend_holiday`

(-0.03), `month` (-0.02), and `day_of_week` (0.01) show linear correlations close to zero with quantity, despite visually evidencing a clear weekly pattern (with Friday as the sales peak). This apparent contradiction is explained by the fact that the Pearson coefficient only detects linear or monotonic relationships, whereas the observed weekly seasonality is cyclical in nature: an increasing numerical value of `day_of_week` (from Monday to Sunday) does not reflect the actual sales pattern, which rises toward Friday and then falls. This result is, in fact, the most direct quantitative justification for the methodological decision to apply cyclical encoding (sine/cosine) to these temporal variables instead of using them in their simple numerical form, linear correlation close to zero does not imply the absence of a pattern, but rather evidences the need for a different representation capable of capturing it.

4.1.8 Demand Intermittency and Sparsity

Across all product-day observations in the training period, 30.22% correspond to zero demand, confirming that the forecasting problem operates in an intermittent-demand regime rather than a continuous one. At the individual product level, mean daily demand ranges from 0.26 to 5.24 pizzas, with a median of 1.27 pizzas across products; several products show mean demand below 0.5 pizzas per day combined with zero-demand rates above 60%. Figure 4.8 shows that this distribution is heavily right-skewed: most products cluster between 0.5 and 2 pizzas per day, while only a small number exceed 3 pizzas per day on average. This context is directly relevant to the interpretation of the error metrics reported in Chapter 4: an MAE of approximately 0.97 pizzas, while numerically small, represents a substantial relative error for products with low mean demand, a characteristic pattern of intermittent-demand forecasting problems stated by Croston [5].

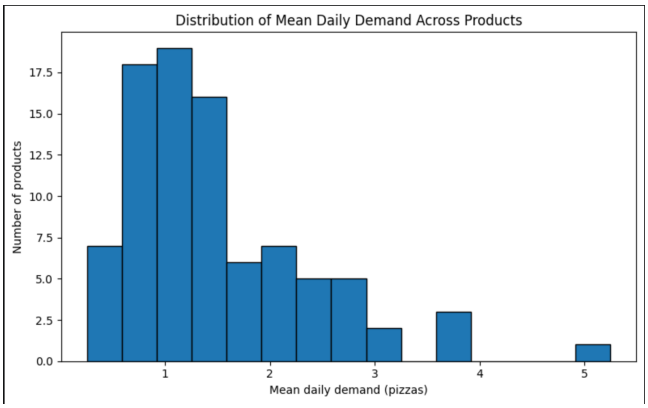


Figure 4.8: Distribution of mean daily demand across products

4.2 Modeling and comparison

To ensure a fair comparison among the four forecasting models, a single data split strategy was used: the last 30 days of the year (December 2015) were reserved as the test set, while the rest of the year was used for training.

All models were evaluated using the same two metrics, expressed in real units (pizzas per day): the Mean Absolute Error (MAE) and the Root Mean Squared Error (RMSE).

4.2.1 LSTM Model

The LSTM model was designed with two independent inputs. The first consists of a 14-day sequence (lookback), composed of eight features: `lag_1`, `lag_7`, `weekend_holiday`, `weather_score`, and the four cyclical encoding variables (`day_of_week_sin`, `day_of_week_cos`, `month_sin`, `month_cos`). The second input corresponds to the product identifier (`item_id`), processed through an embedding layer instead of one-hot encoding, allowing the model to learn a dense representation of each product and capture similarities between products with similar demand patterns. The numerical variables (quantity, `lag_1`, `lag_7`) were normalized using MinMaxScaler before training, as recurrent neural networks require variables

on comparable scales in order to train properly. Since the model relies on a 14-day lookback window per product, the sequences used to evaluate the test period were built using the last days of the training period as historical context, rather than being restricted to sequences built strictly within the test set. This ensures that the LSTM is evaluated on the same 2,730 product-day observations, allowing for a direct and fair comparison across all four models. The model was trained using the Adam optimizer with a learning rate of 0.0005, in batches (batch size) of 32 samples, additionally setting aside 10% of the training set for validation. An early stopping mechanism was incorporated (patience of 5 epochs), which stopped training and restored the weights corresponding to the best observed epoch (epoch 21) with a validation loss (`val_loss`) of 0.0072. After reversing the normalization on the test set predictions, the LSTM model achieved an MAE of 0.93 pizzas, an RMSE of 1.21 pizzas, and a WAPE of 0.66, the lowest error among the four evaluated models.

4.2.2 XGBoost Model

Unlike the previous model, XGBoost does not require variables to be scaled or structured into sequences, but instead operates on a flat table where each row represents a day for a given product. Nine features were used: `item_id` (without special encoding, as tree-based models do not assume linear relationships between numerical categories), `lag_1`, `lag_7`, `weekend_holiday`, `weather_score`, and the four cyclical encoding variables. The model was configured with 200 trees, a maximum depth of 5 levels per tree, and a learning rate of 0.05. These values were not obtained through a formal tuning procedure such as grid or random search, but were set based on configurations commonly reported as effective starting points for gradient boosting on tabular regression tasks in the applied machine learning literature and library documentation. On the test set, XGBoost obtained an MAE of 0.97 pizzas and an RMSE of 1.25 pizzas.

4.2.3 Prophet Model

Unlike XGBoost and LSTM, which share a single model trained on all products simultaneously, Prophet fits an independent model for each time series. In this case, this involved training 91 distinct models (one per product), each configured with annual and weekly seasonality, and incorporating `weekend_holiday` and `weather_score` as external regressors. Prophet obtained an MAE of 1.21 pizzas and an RMSE of 1.62 pizzas, the weakest performance among the evaluated models.

4.2.4 Random Forest Model

As a second tree-based reference model, a Random Forest was trained on the same tabular feature set used by XGBoost (200 trees, maximum depth of 10), in order to isolate whether XGBoost's performance gains stemmed from the boosting algorithm itself or simply from the use of decision trees in general. As with XGBoost, these hyperparameters were set based on commonly recommended configurations for Random Forest regressors reported in the literature, rather than through a tuning procedure specific to this dataset. Random Forest obtained an MAE of 0.98 and an RMSE of 1.26 pizzas, results very close to those of XGBoost and LSTM.

4.2.5 Baseline Models

To contextualize the performance of the four forecasting models described above, two simple baselines were also evaluated on the same test period and the same product-day observations, requiring no training process or hyperparameter tuning. The historical mean baseline predicts, for each product, the average daily demand observed in the training set, applied as a constant value across all days of the test period. This baseline captures the overall typical demand level for each product without accounting for recent trends or seasonality. The seasonal naive baseline predicts, for each product-day, the actual demand observed exactly seven days earlier for that same product, directly reflecting the weekly

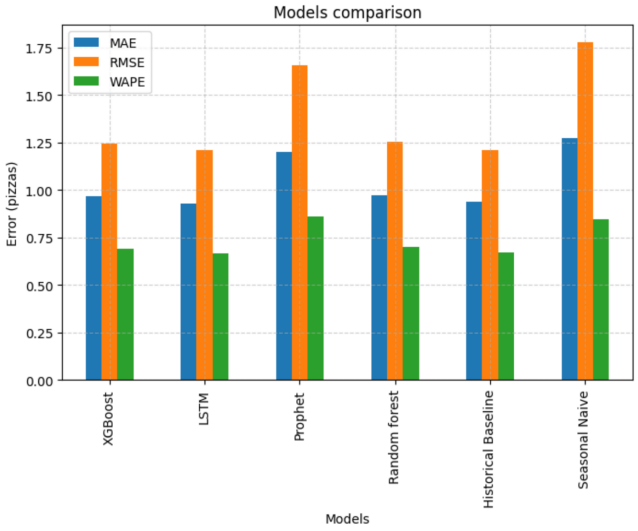


Figure 4.9: Models Comparison

seasonality already identified in the exploratory data analysis. The historical mean baseline obtained an MAE of 0.94 pizzas, an RMSE of 1.21 pizzas, and a WAPE of 0.67, performance comparable to the best-performing machine learning models. The seasonal naive baseline obtained a considerably higher error, with an MAE of 1.27 pizzas, an RMSE of 1.78 pizzas, and a WAPE of 0.85, indicating that recent weekly patterns alone are less informative than the average historical demand level for this particular dataset.

4.2.6 Summary

Figure A.1 summarizes the performance of the six evaluated approaches, showing that LSTM, the historical mean baseline and XGBoost present error levels within a narrow range of one another (MAE between 0.93 and 0.97 pizzas, WAPE between 0.66 and 0.70), while the others show a notably higher error on both metrics.

A relevant finding is that the historical mean baseline, despite requiring no training process and no predictive features beyond a per-product average,

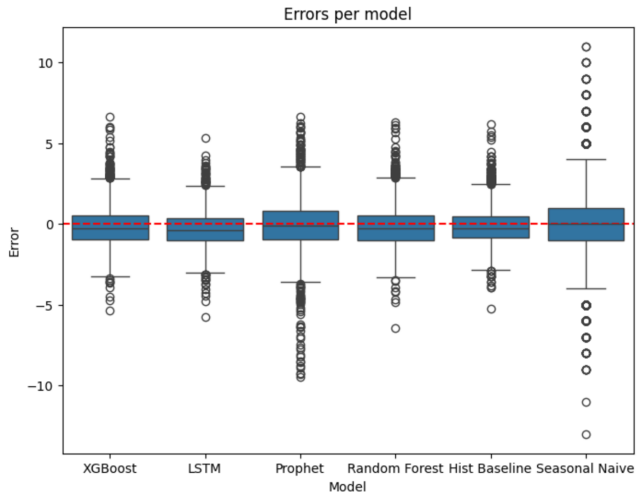


Figure 4.10: Error per model

achieves error levels comparable to the machine learning models evaluated. This suggests that, for this dataset, sophisticated forecasting approaches provide limited improvement over a strong simple baseline. This finding, however, should not be generalized beyond the specific characteristics of this case study: the dataset exhibits a relatively stable, low-volume demand pattern for most products, and it remains unclear whether the same result would hold for a dataset with stronger trends, less intermittency, or higher sales volume per product. Confirming this would require replicating the analysis on a different dataset, which falls outside the scope of this thesis

The analysis of the error distribution per model, Figure A.2 provides an additional nuance that the aggregate metrics do not capture on their own: STM, the historical baseline, XGBoost, and Random Forest show error distributions centered close to zero with similar dispersion, while Prophet and the seasonal naive baseline show a visible bias and considerably greater dispersion; this suggests that their weakness is not limited to a higher average error, but also exhibits less consistent behavior.

Clearly, the best-performing model is LSTM as it achieved the lowest MAE

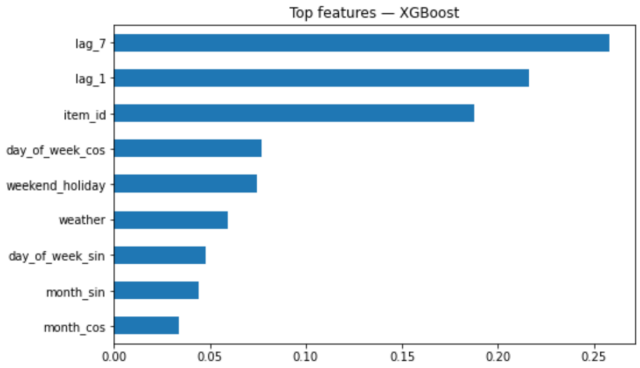


Figure 4.11: XGBoost model top features

among the four machine learning models (0.93 pizzas, compared to 0.97 for XGBoost and Random Forest). But the difference is small enough that, without a formal statistical significance test, it should be interpreted as a similar aggregate error rather than a conclusive superiority of one model over the others. In terms of computational cost, XGBoost completed its training in 0.6 seconds, compared to 4.1 seconds for Random Forest and several minutes for LSTM (trained over 21 epochs). This difference, together with the ability to directly inspect feature importance, an advantage that neural networks do not offer natively, is what ultimately motivated the choice of XGBoost as the system's production model, given its statistically equivalent performance to LSTM at a fraction of the training cost and with considerably greater interpretability. Regarding interpretability, XGBoost's feature importance analysis Figure 4.11 showed that `lag_7` and `lag_1` are the most influential variables, followed by `item_id`. The `weather_score` variable ranked in an intermediate position, below `day_of_week_cos` and `weekend_holiday`, but above the remaining cyclical encoding variables. Although weather showed a practically null linear correlation with sales, the XGBoost model does assign it a non-negligible importance, which is consistent with the idea that its relationship with demand could be nonlinear or conditioned by other variables, rather than nonexistent.

4.3 Forecast and Ingredients

A single day of forecast would not be of much help to the user, since this forecast would be generated once the day's orders are closed, and the forecast would be for the following day. These forecasts follow a recursive approach, that is, the prediction for one day becomes an input for predicting the following day. Specifically, to forecast day +2, the `lag_1` value no longer comes from an actually observed sale, but from the prediction generated by the model itself for day +1. The `lag_7` value, on the other hand, can always be obtained directly from the historical order table, given that, since the horizon is only three days, the date corresponding to seven days prior always corresponds to already observed data. And just as the orders for these horizons are not available, neither is the weather for these days, which is why a neutral `weather_score` value of 3 is applied for days +2 and +3.

4.3.1 Confidence Levels

This recursiveness introduces an important consideration: the prediction error for day +1 propagates, at least partially, to the predictions for days +2 and +3, because these depend on an estimated value rather than an actual one. For this reason, it is necessary to quantify how the error behaves as the forecast horizon extends. To estimate the uncertainty associated with each forecast horizon, the same recursive process was simulated on the test set —where actual values are available—, starting from multiple starting dates throughout that set and recording the error obtained at the 1-, 2-, and 3-day horizons for each starting point. This simulation yielded the following results:

A first relevant finding is that the error does not degrade noticeably as the horizon advances: both the mean error and the standard deviation remain practically stable between day +1 and day +3, with variations of less than 0.01 pizzas in both cases. This suggests that, for a horizon as short as three days, the error propagation described in the previous section does not significantly compromise the quality of the forecast. Based on the mean and standard deviation of each

Horizon	Mean error	Standard deviation
+1 day	-0.196	1.230
+2 days	-0.194	1.229
+3 days	-0.190	1.239

Table 4.2: Mean error and Standard deviation for each Horizon day

horizon, a 95% confidence interval was calculated (approximated as ±2 standard deviations over the error), which produces a margin of approximately ±2.46 pizzas for the three evaluated horizons. This margin is applied to each forecasted value in order to communicate, in addition to the point prediction, an expected range of variation. For example, given a point forecast of 4 pizzas for a given product, the calculated confidence margin allows for establishing an expected range of approximately 1.4 to 6.3 pizzas, information that is incorporated into the dashboard discussed later on.

4.3.2 Ingredients

As established, the same ingredient can be part of many different products in the catalog, which motivates translating product-level forecasts into ingredient-level requirements rather than limiting the analysis to the product level. To achieve this, a recipe catalog was constructed, assigning an assumed base quantity (in grams) to each ingredient present in the 32 distinct pizzas offered, based on standard culinary portions commonly used in the pizzeria industry, since no proprietary recipe data was available for this project. Unlike an approach that applies a single size-based multiplier uniformly to an entire product, this catalog allows each ingredient to scale independently according to its own base quantity, which is then adjusted according to the corresponding product size, using a multiplier that takes size M (medium) as the base unit (S = 0.75, M = 1.00, L = 1.25, XL = 1.50, XXL = 1.75). These ingredient quantities are documented assumptions for this prototype and do not represent verified recipe data from the restaurant in the case study. The detail of how this calculation translates

into the program's final tables and files is documented further ahead.

Chapter 5

Implementation

5.1 Pipeline

The data pipeline was implemented as a sequential flow of six steps. The first step consists of a map & reduce job, and the following five steps are coordinated by a central orchestrator module (orchestrator.py), processing the output of the previous stage, generating the forecast, saving everything to the database, and exporting it to an Excel file. Figure A.3 presents the general data flow diagram: from the ingestion of the raw CSV file, through the calculation of features, the generation of forecasts, up to the final export to Excel, which in turn feeds the dashboard described in the next chapter.

The following sections detail the implementation of each stage in the order in which they are executed.

5.1.1 Orchestrator and Configuration

As already mentioned, from this point onward the pipeline makes use of an orchestrator that is responsible for executing the following steps one by one, as well as passing information from one to another and passing the connection configurations. The entire pipeline operates over a single psycopg2 connection to PostgreSQL. This design avoids the overhead of opening and closing multiple

connections per stage, and allows for the implementation of a single transaction policy for the entire flow. If any step fails, the orchestrator executes a global rollback on the entire connection, reverting any partial writes made by previous stages within the same execution. This design decision prioritizes data consistency over the possibility of persisting partial results; it is preferable not to have a forecast than to have one based on partially corrupted data[7]

5.1.2 Data Ingestion (Step 2)

This step (`step2_load_to_postgres.py`) is responsible for mapping the list generated by the previous step, changing the `pizza_id` identifier from the original file to the system's internal `item_id`, through a join against the `product_catalog` table. This mapping was necessary because the dataset's original identifier did not correspond directly to the relational schema designed for the project. Once this change is ready, a column is added to indicate which day these orders correspond to, for now, it is configured to be the day being processed, since it is assumed that this pipeline will run on the same day the original orders file is generated. Once these three columns are ready, and after verifying that they are in the required format, they are inserted into the PostgreSQL database line by line.

5.1.3 Feature Engineering and Forecasting (Steps 3-4)

Steps 3 and 4 of the pipeline are responsible for calculating the features used by the forecasting model. These include lag features, specifically `lag_1` (previous day's value) and `lag_7` (value from the same date the previous week), cyclical encoding of temporal variables (day of the week, month), and the `weather_score`, a composite index that combines temperature and precipitation on a scale of 0 to 5. The difference between these steps is that step 3 (`step3_compute_features.py`) is responsible for performing the feature calculation for the data that was recently uploaded to the database, since this data does allow for a more specific weather score to be calculated, following the

methodology implemented in the Jupyter notebook. Step 4 is responsible for calculating the features for forecast days +2 and +3; the `weather_score` value is kept fixed at 3 (neutral value), as no actual weather forecast is available for those days, and it likewise checks the orders table in the database in order to obtain `lag_1` and `lag_7`. This step is also responsible for running the XGBoost forecasting model through a loop based on the number of days set in the `FORECAST_DAYS` variable, three in this case.

5.1.4 Forecast PostgreSQL Table and Final Excel Output (Steps 5-6)

Step 5 (`step5_forecast_table.py`) follows a series of steps that basically drop the forecast table, recreate it, and insert the output generated from the previous step. This makes it possible to remove forecast days that have already passed, always keeping the forecast from today to three days ahead. The final step 6 (`step6_excel_sheets.py`) is responsible for generating an Excel file that contains two sheets. The first sheet basically contains the forecast table plus three months of historical orders; although the dashboard does not use the three months, it is always good to give the client a window in which they can perform some analysis. The second sheet shows, per day, the quantity of ingredients in units that will be consumed. In the same way that this sheet was created in the Jupyter file, the `product_catalog` table is queried, which stores the ingredients of each product as a comma-separated text field. In order to perform calculations at the individual ingredient level, this column is exploded, in a process that separates each ingredient into its own record, associated with the corresponding product. Using the forecasted quantity of units per product and size, the requirement for each ingredient is obtained by multiplying the forecasted quantity by the ingredient-specific gram quantity defined in a recipe catalog (scaled according to product size). This recipe catalog is maintained as an editable reference file, separate from the pipeline code, so that ingredient quantities can be adjusted without modifying the underlying implementation.

5.1.5 Summary

The initial methodological proposal considered the use of Apache Kafka for real-time data ingestion. However, during implementation, it was confirmed that the restaurant delivers sales information in CSV files on a daily basis, not as a continuous stream of events. Consequently, a batch processing approach was chosen, which considerably simplified the architecture without compromising the system's objectives, because the use case does not require real-time forecasts. At first, the plan was to calculate the amounts of the ingredients and store them in PostgreSQL; however, the calculations were to be carried out entirely in memory, with the results being exported directly to the Excel file, since there was no situation that called for storing the data persistently in the database, the dashboard taking the exported file directly. The pipeline described in this chapter was implemented and validated with a successful end-to-end run, from the ingestion of the raw CSV file to the generation of the final daily_sales_forecast.xlsx Excel file.

5.2 Dashboard

As a final presentation, an interactive dashboard was built, responsible for translating the results generated by the pipeline into a visual tool usable by the end user, without requiring technical programming or data analysis knowledge. It was developed using Streamlit as the main framework, together with Plotly for the generation of interactive charts. The dashboard has two pages. The first, `pizzas_forecast.py`, functions as the entry point and presents the sales forecast. The second, located at `pages/Ingredientes_table.py`, is a secondary page dedicated to ingredient requirements. This organization takes advantage of Streamlit's native multi-page routing system, which automatically detects files within the pages/ folder and generates the corresponding navigation without requiring additional configuration. As already mentioned, the previously explained pipeline generates an Excel file as output; the dashboard consumes that

file, removing the need for a direct connection to the PostgreSQL database. Instead, it exclusively consumes the `daily_sales_forecast.xlsx` file with its two sheets (`sales_last3months_forecast` and `ingredient_requirements`). This allows the pipeline to be run as many times as necessary independently, so that the dashboard can continue consuming the output regardless of whether the other one crashes, bringing in the most recent result each time a user opens it.

5.2.1 Configuration

As already mentioned, the dashboard has two executable Python pages, but it also has two other files: `requirements.txt`, which contains the names of the libraries required to run this dashboard (streamlit, pandas & plotly). The second file is `data_loader.py`, which is used by both pages to read the information from the Excel file and load the current date, showing which day this dashboard was run. The functions `load_sales_data` and `load_ingredients_data` are responsible for reading the Excel file; each one reads the corresponding sheet from the file and stores it in cache so that the action does not have to be repeated unless there is some modification.

5.2.2 Page 1

Main page that opens when the program is run. It has five sections, each with a chart that helps the client read the insights obtained from the generated forecast. The first section has three cards that show the total predicted number of items to be sold per forecast day, along with the dates and a small comparative analysis versus the previous day. The second section is a line chart that shows the trend followed over the past seven days of sales plus the three forecast days, each part marked with a different color and line type to indicate which are the predictions. For the third section, there is a stacked bar chart that shows the total item sales by variant, accumulated for the forecasted days; for this example, these are the total pizza sales by size. The fourth section is a prediction for the following day (first forecasted day) of the top three items by variant; for this example, these

are the top three pizza flavors by size. And finally, there is a card that shows the total number of items to be sold for tomorrow compared to sales from seven days ago, which can be modified through two filters, variant and item.

5.2.3 Page 2

Second page of the dashboard, dedicated to the ingredient requirements derived from the forecast. It has two sections, both built around the same stock coverage calculator. The first section is a small editable table where the user types how much stock they currently have of each ingredient, in grams. The second section is a table that takes that input and, for each ingredient, subtracts the forecasted requirement day by day, carrying over any stock left from one day into the next, to show how much is still missing to cover each of the three forecast days, along with the total missing amount across the whole period; a value of zero means the available stock is enough to cover that day's demand, while any value above zero is the amount still needed.

5.2.4 Summary

The initial methodological proposal considered using Tableau Public and Google Sheets (through the gspread library) as the visualization layer. During implementation, the decision was made to migrate to Streamlit in order to avoid depending on external services that may require a subscription fee, making this service accessible to smaller businesses. This decision also made it possible to integrate custom interactive components, such as the editable stock table on page two, which would have been more difficult to implement in the originally proposed tools. The dashboard described in this chapter closes the complete cycle of the system, transforming raw sales data, processed and forecasted throughout the previous stages, into an actionable visual tool for the restaurant's administrative staff.

Chapter 6

Testing and Evaluation

The system developed throughout this thesis covered the complete cycle of a demand forecasting pipeline: from the ingestion and cleaning of approximately 48,000 historical orders, through the identification of seasonality patterns and ingredient complexity, the comparison of four modeling approaches, up to the generation of a three-day recursive forecast with associated confidence levels and its translation into ingredient requirements. This section jointly discusses the most relevant findings from these stages, directly answering the research questions posed.

6.1 Weather

One of the questions was whether weather directly affects sales. As was verified, at the aggregate monthly level, no clear relationship was observed between the weather index and the total order volume: months with very different weather conditions (average score of 1.3 in winter versus 4.6 in summer) showed similar sales volumes, within a relatively narrow range of 3,900 to 4,400 monthly orders. This absence of a relationship was confirmed quantitatively through the Pearson correlation coefficient between weather and quantity at the daily level, which was practically null. However, the feature importance analysis of the XGBoost

model leads to this conclusion: weather was not the least relevant variable in the model, ranking above three of the four cyclical encoding variables used. This suggests that, although weather does not exhibit a direct linear relationship with sales at the aggregate level, it could provide marginal, nonlinear information to the model when combined with other variables (for example, day of the week or season), rather than constituting a strong predictor on its own. The fact that this contradiction arises is similar to a pattern that was seen with the `day_of_week` variable, a variable which also displayed a linear correlation near zero with respect to sales even though it clearly showed a weekly pattern (Figure 4.2). In that instance, it was explained that the Pearson coefficient only picks up linear or monotonic relationships, the kind of pattern being cyclical. A somewhat related but not the same explanation holds for the weather: the linear correlation coefficient considers the variable by itself, whereas a tree-based model such as XGBoost is able to capture conditional or interaction effects, meaning that weather might only become relevant when combined with other variables rather than on its own. Together, these two examples show a more general methodological point for this thesis: the lack of a linear correlation should not be taken to mean that there is no predictive value, but instead should be seen as a sign that a different type of relationship—cyclical or conditional—may be at play and thus require a modelling approach capable of detecting it. Weather measured as a composite index of temperature and precipitation, has a weak, nonlinear influence on pizza demand in the analyzed dataset, since it indicates that other factors (particularly recent past behavior, captured by `lag_1` and `lag_7`) are considerably more determinant for the forecasting of this type of product.

6.2 Models

The comparison of the six evaluated approaches, including two simple baselines, showed that LSTM, the historical mean baseline, XGBoost, and Random Forest achieved similar aggregate error (MAE between 0.93 and 0.97 pizzas, WAPE

between 0.66 and 0.70), while Prophet and the seasonal naive baseline showed considerably higher error (MAE of 1.20 and 1.27, respectively) along with more biased and dispersed error distributions. This result is consistent with the central argument introduced in Chapter 1: that the choice of forecasting algorithm is not, by itself, the primary factor determining whether a system can effectively reduce food waste. The small margin of error between the historical baseline and the more advanced machine learning models indicates that, with respect to this dataset, the particular modeling technique makes only a minor difference once a certain point is reached; this supports the view that the more important design choice concerns the accuracy with which a product-level forecast—no matter where it comes from—is converted into ingredient-level requirements. To put it plainly, simply choosing a different model on its own could not have been enough to overcome the gap between forecasting and purchasing decisions which was identified as the main reason for this study; it is the translation process described in Chapter 5, not the underlying algorithm, that finally determines the practical value of the forecast for reducing waste. Despite LSTM achieving the lowest MAE among the four machine learning models, XGBoost was selected as the production model, not on the basis of the error metric, but as a result of a combined assessment of its performance, computational cost (a training time of 0.6 seconds compared to several minutes for LSTM), and interpretability (since feature importance analysis can be used, something which is not natively available in a recurrent neural network). This choice is in line with the principle that in a system designed for actual use by non-technical personnel, the ability to explain why the model reaches a particular prediction has practical value that can justify a small, statistically unconfirmed difference in accuracy.

6.3 Practical Implications of Ingredient-Level Requirement Estimation

The findings regarding the high reuse of ingredients across products in the catalog (for example, garlic present in 57 of the 91) is why translating product-level forecasts into ingredient-level requirements carries practical relevance, rather than limiting the analysis to the product level. A relatively small forecasting error, poorly distributed across several products that share the same ingredient, could, in principle, be amplified or offset in ways that are not obvious if purchasing planning were based solely on manually aggregated product forecasts. The analysis of confidence levels by horizon showed that the forecast error remains stable between the 1- and 3-day horizons (variations of less than 0.01 pizzas in mean error and standard deviation). The obtained 95% confidence margin (approximately ± 2.46 pizzas) was incorporated as an additional layer of information in the system, allowing not only a point figure to be communicated, but also an expected range of variation, a practice that is valuable in the context of input planning, where underestimating demand has a different cost (stockout) than overestimating it (waste).

6.4 Summary

Several limitations must be considered when interpreting the results of this thesis. First, the dataset used corresponds to a single year (2015) and a single establishment, which restricts the ability to generalize the findings, specially the conclusion regarding the weak influence of weather, to other geographic contexts, restaurant types, or longer time periods. Second, the use of a fixed, neutral value (3 out of 5) for the weather variable on days +2 and +3 of the forecast horizon, due to the lack of an actual weather forecast available for those dates, introduces a source of error that is not specifically quantified separately from the rest of the model. Although the analysis showed that the total error does not degrade noticeably between horizons, it is not possible to fully isolate

how much of that stability is due to the generally low influence of weather versus a possible compensating effect among variables. Even though the XL and XXL sizes have been excluded from the training set for reasons relating to their small volume (1.2% of orders), the system fails to produce specific forecasts for these sizes. Additionally, the ingredient quantities used to translate product-level forecasts into ingredient-level requirements are based on assumed culinary portions rather than verified recipe data from the restaurant, since no proprietary recipe information was available for this project. While this does not affect the relative comparison between forecasting models, it does affect the absolute accuracy of the ingredient requirements reported by the system and should be replaced with verified data before any real-world deployment. Finally, the finding that a simple historical mean baseline achieves error levels comparable to the machine learning models evaluated should be interpreted as specific to the characteristics of this dataset, namely its relatively stable, low-volume, and intermittent demand pattern at the individual product level.

Chapter 7

Conclusion

This thesis addressed the problem of food waste in the restaurant sector from the perspective of demand forecasting, proposing and implementing a complete system capable of anticipating not only the sales volume of a product, but also the specific requirements of each ingredient that composes it. Throughout this work, an end-to-end data pipeline was designed, built, and validated, different machine learning approaches were compared for the forecasting task, and a visualization tool oriented toward operational staff without technical knowledge was developed. The results obtained allow us to conclude that a system of this type is technically feasible with moderate tools and data volumes, without requiring specialized computing infrastructure. The comparison between XGBoost, LSTM, Random Forest, and Prophet showed that a gradient boosting model can achieve error levels similar to those of a recurrent neural network in this task, at a fraction of its computational cost and with the additional advantage of offering direct interpretability regarding which variables influence each prediction, a valuable characteristic in a system intended to be used, and eventually trusted, by non-technical staff. A further finding was that a simple historical mean baseline, requiring no training process, achieved error levels comparable to the machine learning models evaluated, while a seasonal naive baseline performed considerably worse. This finding in no way undermines the contribution

of machine learning methods, since it supports one of the main points of this thesis—that the selection of forecasting algorithm alone is not the factor which determines whether a system is able to effectively reduce food waste. This finding is specific to the low-volume, intermittent demand pattern observed in this dataset, and should not be assumed to generalize to other contexts without further evaluation. Likewise, the work showed that the incorporation of weather variables, although supported by the literature as a potentially relevant feature in food consumption behavior, had a weak, nonlinear influence in the analyzed case study. This finding highlights the importance of grounding design decisions in empirical evidence specific to the context of each implementation, rather than generally assuming that a factor reported as relevant in another study will behave the same way in any scenario. Finally, the finding that a small number of ingredients is repeated across a considerable proportion of the product catalog confirms the practical relevance of translating product-level forecasts into ingredient-level requirements: this granularity allows for more precise purchasing planning than what would be obtained by manually aggregating forecasts made only at the product level. This granularity was further supported by building a recipe catalog that assigns independent quantities to each ingredient, replacing an earlier, coarser approach based on a single per-product size multiplier.

7.1 Future Work

Extending the dataset to multiple years and, if possible, to multiple establishments, would make it possible to evaluate whether the conclusions obtained (like the weak relationship between weather and sales) hold in different temporal and geographic contexts, or whether they are specific to the analyzed case study. The structured persistence of ingredient requirements in the PostgreSQL database, discarded during implementation in favor of a direct export to Excel, could be considered if the system evolves toward a use case that requires historical traceability of the generated requirements, or its integration

with other restaurant systems, such as an inventory management module or a purchase order module for suppliers. A natural extension tied to the previous point would be to migrate the pipeline described from its current single-node implementation to a distributed execution engine such as Apache Spark, which would become necessary once the system is extended to multiple restaurant locations simultaneously rather than a single case study. This migration would primarily affect the execution backend of Steps 2 through 4 (ingestion, feature engineering, and model training) while the relational schema, the recipe catalog, and the ingredient-requirement translation logic described in this thesis would remain conceptually unchanged. Additionally, revisiting a streaming ingestion approach (such as the Kafka-based design initially considered) would become relevant again in a multi-location deployment where point-of-sale systems generate a continuous stream of orders across many sites, rather than a single daily batch file per location. Finally, it would be worth exploring whether a differentiated treatment for the sizes excluded from training due to their low volume would provide additional practical value, either through a separate, lower-complexity model or through simple heuristic rules based on their historical proportion relative to other sizes. This research demonstrates that the combination of a well-designed data pipeline, an interpretable forecasting model, and an accessible visualization tool can create a practical and viable solution for reducing food waste in restaurants, without requiring complex technological infrastructure or technical knowledge on the part of the staff who ultimately use the system in their daily operation.

Bibliography

[1] L. Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.

[2] H. Chan and M. I. M. Wahab. A machine learning framework for predicting weather impact on retail sales. *Supply Chain Analytics*, 5:100058, 2024.

[3] C. Chatfield. Calculating interval forecasts. *Journal of Business & Economic Statistics*, 11:121–135, 1993.

[4] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, pages 785–794, New York, NY, USA, 2016. ACM.

[5] J. D. Croston. Forecasting and stock control for intermittent demands. *Journal of the Operational Research Society*, 23:289–303, 1972.

[6] Robert Fildes, Shaohui Ma, and Stephan Kolassa. Retail forecasting: Research and practice. *International Journal of Forecasting*, 38(4):1283–1318, 2022.

[7] J. Gray. The transaction concept: Virtues and limitations. pages 144–154, 1981.

[8] C. Guo and F. Berkhahn. Entity embeddings of categorical variables. *arXiv preprint arXiv:1604.06737*, 2016.

[9] J. Gustavsson, C. Cederberg, U. Sonesson, R. van Otterdijk, and A. Meybeck. Global food losses and food waste – extent, causes and prevention. Technical report, 2011.

[10] M. S. Hossain and F. Parvin. A comparative study of various statistical and machine learning models for predicting restaurant demand in bangladesh. *PLOS ONE*, 20:e0325449, June 2025.

[11] R. J. Hyndman and A. B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22:679–688, 2006.

[12] N. Kumar, K. Dheenadayalan, S. Reddy, and S. Kulkarni. Multimodal neural network for demand forecasting. *arXiv preprint arXiv:2210.11502*, 2022.

[13] J. Leites, V. Cerqueira, and C. Soares. Lag selection for univariate time series forecasting using deep learning: An empirical study. *arXiv preprint arXiv:2405.11237*, 2024.

[14] Nathan Marz and James Warren. *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications, Shelter Island, NY, 2015.

[15] S. Mohammed, L. Budach, M. Feuerpfeil, N. Ihde, A. Nathansen, N. Noack, H. Patzlaff, F. Naumann, and H. Harmouch. The effects of data quality on machine learning performance on tabular data. *arXiv preprint arXiv:2207.14529*, 2022.

[16] K. Posch, C. Truden, P. Hungerländer, and J. Pilz. A bayesian approach for predicting food and beverage sales in staff canteens and restaurants. *arXiv preprint arXiv:2005.12647*, 2021.

[17] S. Siami-Namini, N. Tavakoli, and A. S. Namin. A comparison of arima and lstm in forecasting time series. pages 1394–1401, 2018.

[18] S. J. Taylor and B. Letham. Forecasting at scale. *The American Statistician*, 72:37–45, 2018.

[19] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. Apache spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65, 2016.

[20] H. J. Zietsman and J. H. van Vuuren. A generic framework for decision support in retail inventory management. *arXiv preprint arXiv:2207.13923*, 2022.

Appendix A

Appendix

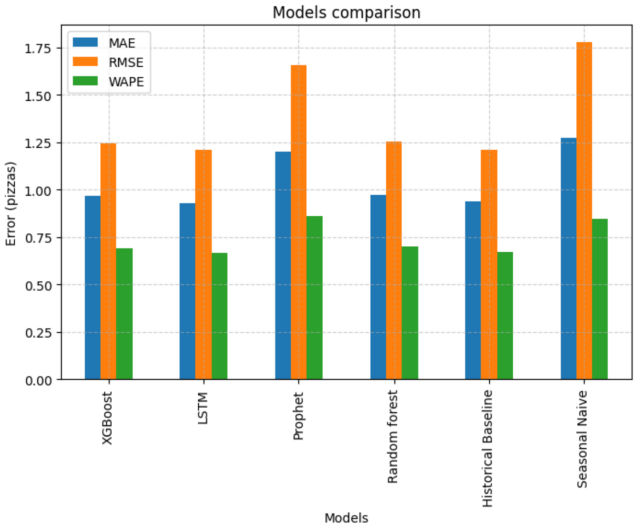


Figure A.1: Models Comparison

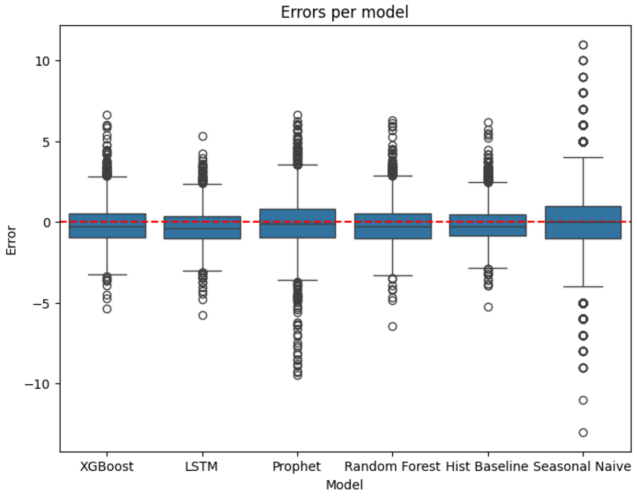


Figure A.2: Error per model

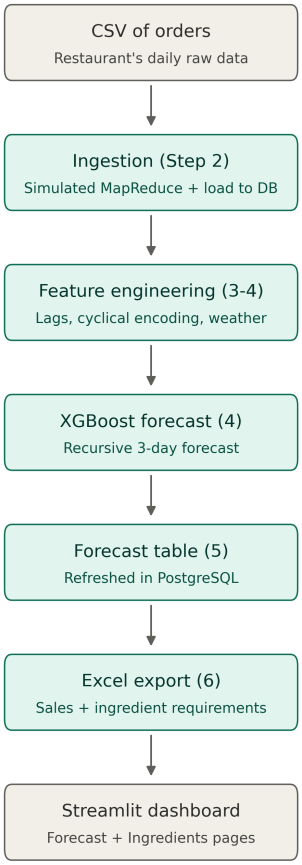


Figure A.3: Data pipeline flow, from raw order ingestion to the interactive dashboard.